

EECE 4830-5830 Network Design, Dr. Vinod Vokkarane

Programming Project Phase 4: Implement Go-Back-N protocol over an unreliable UDP channel

Project description: Project description: The TCP/IP stack has five layers, namely application, transport, network, link, and physical. In Phase 1, each group implemented the standard user datagram protocol (UDP) sockets. The intention was to transfer a file between a UDP client process and a UDP server process. In Phase 2, Phase 3, we provided reliable data transfer service over the unreliable UDP connection. In Phase 4, you will have to enhance your code to handle pipelined data transfer. The sender has to maintain a buffer as shown below.

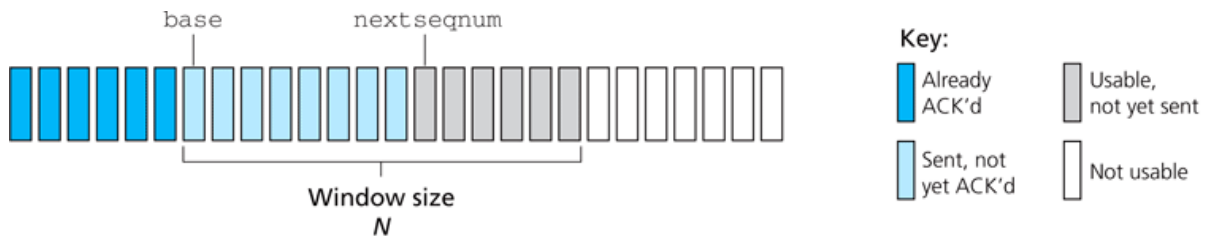


Figure 3.19 ♦ Sender's view of sequence numbers in Go-Back-N

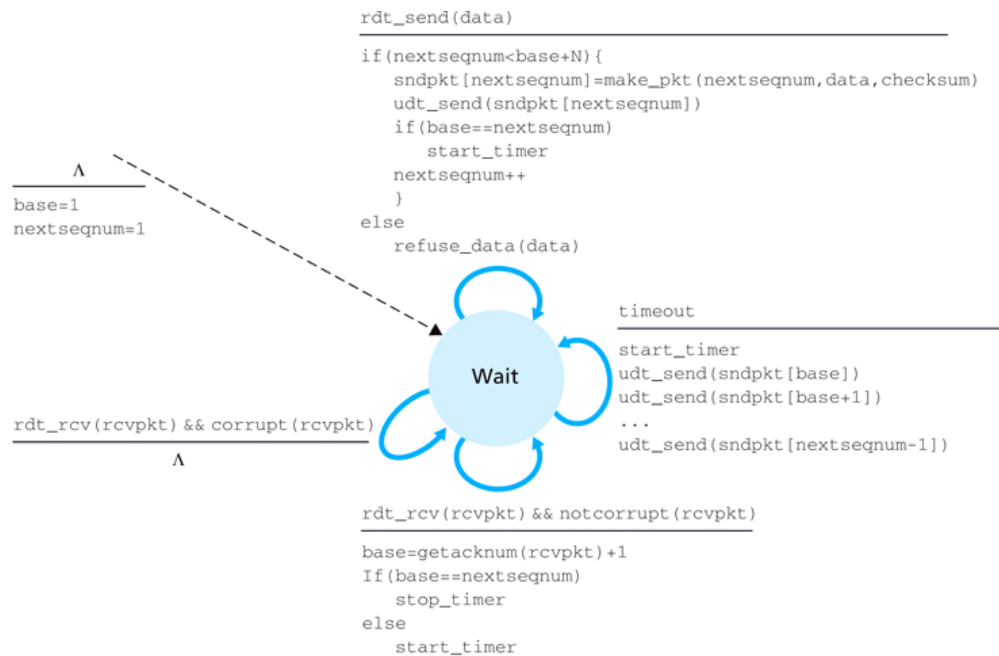


Figure 3.20 ♦ Extended FSM description of GBN sender

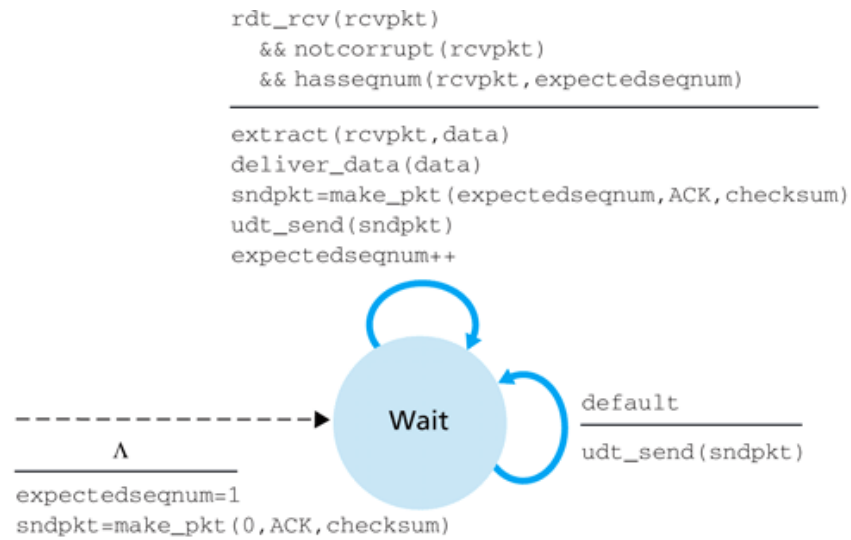


Figure 3.21 ♦ Extended FSM description of GBN receiver

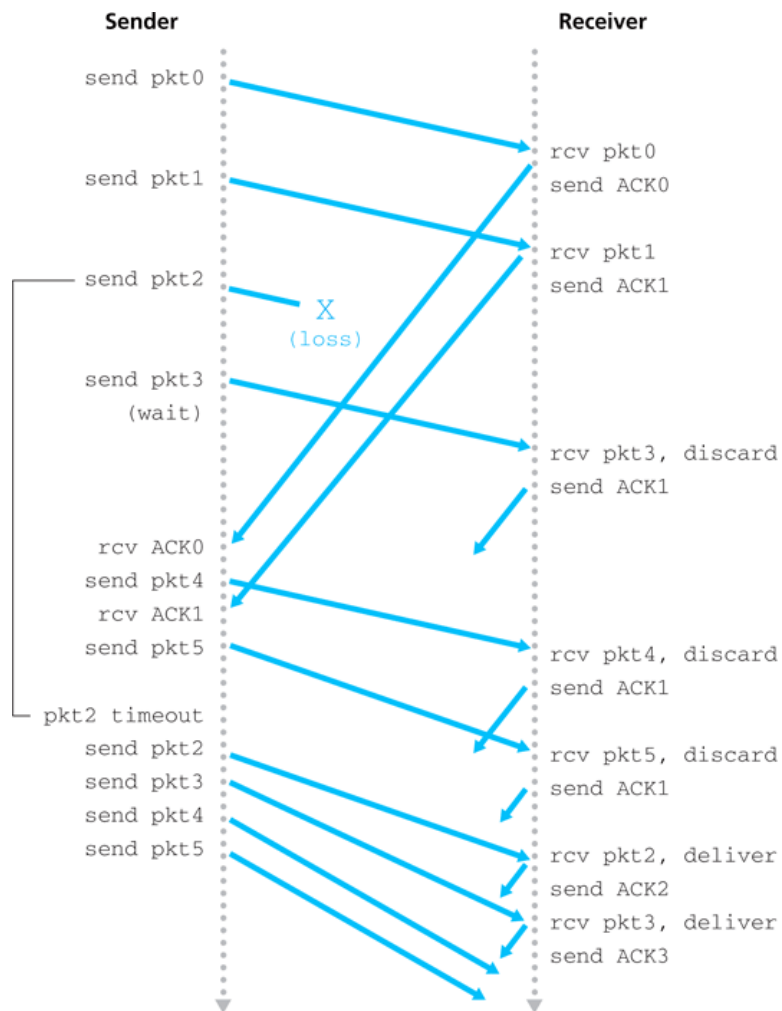


Figure 3.22 ♦ Go-Back-N in operation

The following are the basic implementation steps:

1. Pick a transfer file.
2. *Make_packet* function - parses the image file and breaks it down to several packets (set a fixed packet size, say 1024 bytes).
3. Set a fixed window size (N).
4. Use the UDP sockets (From previous phases) to send and receive GBN packets. UDP sockets are unreliable, so they simulate the unreliable Internet IP (network) layer.
5. Implement: Sequence Numbers (to identify duplicates), Checksum (implement your own, similar to UDP), ACKs, and a countdown timer (to handle packet loss).
6. The GBN receiver should assemble packets in order and deliver the entire transfer file to the application.

In this phase, you will need to implement different data transfer scenarios

Option 1 - No loss/bit-errors.

Option 2 - ACK packet bit-error: you need to intentionally change the data bits of the received ACK packet at the sender (some error %) and implement suitable recovery mechanism.

Option 3 - Data packet bit-error: you need to intentionally change the data bits of the received DATA packet at the receiver (some error %) and implement suitable recovery mechanism.

Option 4 - ACK packet loss: you need to intentionally drop the received ACK packet at the sender (some loss %) and implement suitable recovery mechanism.

Option 5 - Data packet loss: you need to intentionally drop the received DATA packet at the receiver (some loss %) and implement suitable recovery mechanism.

IMPORTANT: You should be able to demo Phase 4 with and without loss recovery.

Performance Graphs

Note:

- Don't set your timeout to be too high.
- Each point on the plot should be an average of multiple runs at a given loss/error rate.
- For accuracy remove print commands and debug statements to the output console statements while measuring the completion time for the chart.

Plot charts for completion time for all your 5 options for transferring the same file at 0% loss/error to 95% loss/error in increments of 5%.

Chart 1: Phase 4 performance

- X-axis: Intentional loss probability (0% -95% range)
- Y-axis: File Transfer Completion Time

Chart 2: Phase 4 performance with a fixed 10% loss probability

- X-axis: Window Size (1, 2, 5, 10, 20, and 50)
- Y-axis: File Transfer Completion Time

Chart 3: Performance comparison of the different phases at a fixed loss probability (say 10%) and a fixed bit-error probability and using the same transfer image

- X-axis: Phase 1, Phase 2, Phase 3, Phase 4
- Y-axis: File Transfer Completion Time

Observe and conclude if there is an optimal window size and an optimal timeout value for your project.

Programming language: Python.

Deliverables:

1. DESIGN_DOCUMENT.pdf/md

1. If submitting on BlackBoard, it MUST be a PDF
2. If you only want to submit your GitHub repo, it can remain a MD

2. README.pdf/md

3. Plot charts for completion time for all your 5 options for transferring the same file at 0% loss/error to 95% loss/error in increments of 5%.

4. Individual Contribution (contribution.txt/doc):

This is confidential submission for each member

References:

1. Socket Programming
 - [JAVA Socket Programming](#)
 - [Linux Gazette's Socket Programming](#)
 - <http://www.programminglogic.com/sockets-programming-in-c-using-udp-datagrams/>
 - [Socket Programming by JAVA World](#) and [O'REILLY JAVA Network Programming](#)
 - [Python Socket Programming](#)
2. UDP: [RFC768](#)
3. Principles of Reliable Data Transfer, Section 3.4.1, Page 206 -207, Kurose/Ross (6th Ed).